Modular Monolith

microservice discipline, no network

Watch the 4-min video

In one line: a modular monolith is one deployable split into modules with strong, **enforced** boundaries: each module owns its logic and tables and exposes a small public API, so you get clean code, in-process calls and real transactions now, and an easy extraction path later.

1 Microservices from day one?

A startup has six engineers and one product, and someone proposes microservices. That buys network calls, many databases and a lot of operations work before there is any team bottleneck to solve (lesson 19). The middle path is the **modular monolith**: the discipline of service boundaries without the distributed system.

2 One deployable, strong modules

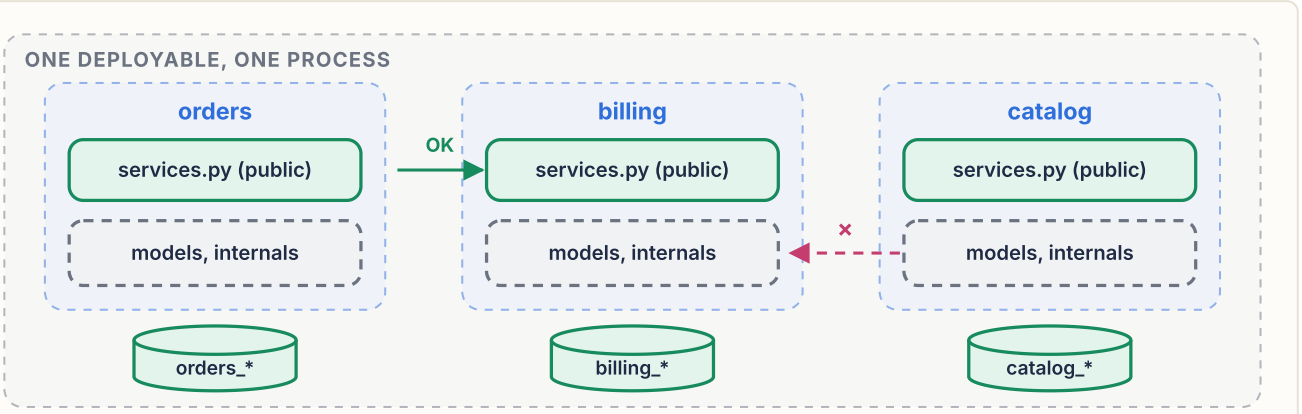


Figure 1. Modules talk only through each other's public **services** API; internals and tables are private.

Inside the single deployable, code is split by business capability: orders, billing, catalog. Each module:

- exposes a **small public interface** (a facade or `services` module of functions);
- keeps everything else private: models, helpers, its own tables (own schema or table prefix);
- is called with plain function calls, so calls are fast and can share **one database transaction** when they truly must.

Treat each module as a service that happens to live in the same process. Because its API is already narrow, it can later become a network API (REST or gRPC) with little change.

3 Enforce the boundaries

Boundaries that are only conventions decay into a **big ball of mud** within months. Enforce them with tooling:

- **Import rules in the build:** import-linter (Python), ArchUnit (Java), Packwerk (Ruby, built by Shopify), or separate projects/assemblies in .NET.
- **Architecture tests** that fail when one module queries another module's tables.

- **Per-module schema** or table prefix, ideally with separate database users so a stray query fails.
- **Code ownership** (e.g. a CODEOWNERS file) so each module has a responsible team.

Shopify runs one of the largest Ruby on Rails monoliths this way, with components and Packwerk checks rather than microservices.

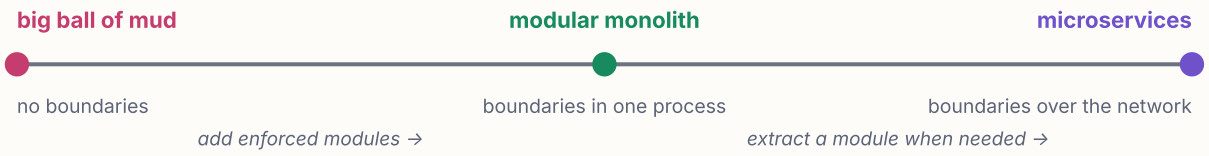


Figure 2. The modular monolith sits in the middle: real boundaries, still one process.

4 Why it suits a startup, and its limits

	Modular monolith	Microservices
Deploy	one pipeline, one artefact	one pipeline per service
Calls between modules	function calls	network calls (timeouts, retries)
Transactions	real ACID when needed	sagas, eventual consistency
Fault isolation	none: one leak hurts all modules	per service
Scaling	whole app only	per service
Release coupling	one bad change blocks the release	teams release independently

The bottom three rows are the real limits. They are usually fine early, and they tell you **when to extract** a module: a scaling hot spot, a module that needs its own release schedule or team, or very different runtime needs. Extracting a clean module is far easier than untangling a messy monolith.

WORKED EXAMPLE: DJANGO APPS WITH IMPORT RULES

A Django project has apps `orders`, `billing` and `catalog`. Each app exposes a `services.py`; everything else is internal. An import-linter contract in CI forbids `billing` from importing `orders.models`; it must call `orders.services.get_order_total(order_id)`. Each app keeps its own tables (Django already prefixes them, e.g. `orders_order`). When billing later gets its own team and release schedule, its calls to orders are already narrow, so they become HTTP calls and billing is extracted with few changes.

COMMON MISTAKES

- Boundaries by convention only, with no build rule or test to stop violations.
- Modules joining across each other's tables "just this once".
- Splitting by technical layer (controllers, models) instead of business capability.
- Extracting a module with no real reason, getting the network costs without a benefit.

5 Key terms

Modular monolith: one deployable with enforced internal module boundaries

Module API: small public interface; the rest is private

Big ball of mud: code with no real boundaries

Architecture test: automated test of dependency and data rules

Import rule: build check that forbids imports across module internals

Extraction: turning a module into a separate service

6 Interview questions

Q1. What is a modular monolith? BASIC

One deployable unit whose code is split into business modules with strong boundaries: each owns its logic and tables and is used only through a small public API.

Q2. What matters most to make it work? BASIC

Enforcing the boundaries with tooling (import rules, architecture tests, per-module schemas), not just conventions.

Q3. Why might a modular monolith be better than microservices for a startup? SCENARIO

It keeps one simple deployment, fast in-process calls and real transactions, while module boundaries keep the code clean; a module can be extracted later when a real need appears.

Q4. When would you extract a module into a service? SCENARIO

When it has a scaling hot spot, needs its own team and release cadence, has different runtime or compliance needs, or its failures must be isolated.

Q5. Two modules need to update data atomically. Is that allowed? TRICKY

Yes, through public APIs in one transaction, which is a key advantage; but note it as coupling that will need a saga if they are ever split.

Q6. What can a modular monolith not give you? TRICKY

Separate scaling, fault isolation (a memory leak hurts every module) and independent releases, because it is still one process and one deployment.

Next: SD-L-003 · Domain-Driven Design Basics backendroadmap.com